StudyBuddy - Group 10 - Iteration 3 Log

Members:

- Sean Lee-Wah - 221344163 - sean1010@my.yorku.ca
- Bhavya Trivedi - 219743590 - Bhavya23@my.yorku.ca
- Aqsa Malik - 221100987 - xsa@my.yorku.ca
- Vaughn - 219846914 - vc8@my.yorku.ca
- Omar Fakousa - 220609426- omar821@my.yorku.ca
- Yash Bhupta - 219743285 - bhupta01@my.yorku.ca

Updated External Links:

Jira: https://le-eecs-2311-project.atlassian.net/jira/software/projects/ID/boards/1

Architecture Diagram (Updated): 3.0. Architecture · hvpham-yorku/project-group-10_StudyBuddy Wiki

ITR3 Meeting Summary Video: 🎬 Stakeholder Meeting 2

Final Working Branch for Iteration 3: Click Here

Meeting Minutes: March 15, 2026 (72 min) Attendees: Bhavya, Vaughn, Yash, Omar, Sean, Aqsa

Summary We kicked off Iteration 3 by auditing our current Firebase usage and security vulnerabilities. The primary goal for this final phase is moving from a functional MVP to a "production-ready" state. This involves optimizing our database costs, which have spiked due to chat polling, and implementing mandatory security features like 2FA and user reporting. We also discussed the integration of campus-specific location data to make the map feature more useful for York University students.

Key Decisions

- **Performance Optimization:** Agreed to refactor the chat retrieval logic. Instead of full document reads on every poll, we will implement a "lightweight check" strategy to see if new messages exist before fetching data.

- **Security Protocol:** 2FA is now a requirement for the "Secure" version of the app. We will use Gmail-based SMTP for code delivery.
- **Moderation:** Established a "User Reporting" system and a "Privacy Policy" restriction (ID 95) where only joined members of a session can view or participate in specific event chats.

Task Assignments

- **Bhavya:**
  - Limit Firebase Reads/Writes (ID 100) by implementing timestamp-based polling for chat.
  - Perform Auth Cleanup (ID 101) to standardize token usage across all API calls.
- **Vaughn:**
  - Implement Event Review Details and Comments (ID 92, 94) for past sessions.
  - Handle "Event Restrictions" so only joined users can access sensitive session data (ID 95).
- **Yash:**
  - Build the Auto-Timeout feature (ID 98) with a user-configurable timer in settings.
  - Implement 2FA and Password Reset logic (ID 104) integrated with Firebase Auth.
- **Sean:**
  - Implement "Display Current Location" (ID 46) and calculate the nearest campus building using the York U coordinates CSV.
  - Build the "Study Streak Counter" (ID 102) to track user engagement.
- **Aqsa:**
  - Develop the Map View for possible study sessions (ID 33) and the internal navigation logic (ID 48).
- **Omar:**
  - Design the "User Reporting System" (ID 97) for flagging inappropriate behavior.
  - Harden the Firestore rules for a "Secure Database" (ID 62) to prevent unauthorized read/writes.

Meeting Minutes: March 18, 2026 (55 min) Attendees: Bhavya, Vaughn, Yash, Omar, Sean, Aqsa

Summary This mid-iteration sync focused on technical blockers regarding location accuracy and session persistence. We reviewed Sean's progress on building detection and decided to use the Euclidean algorithm for proximity calculations. Yash raised concerns about whether a page reload should trigger a logout after a timeout, and we agreed that the token should be cleared from local storage entirely. Bhavya confirmed that the chat optimization has successfully reduced read costs by approximately 80%.

Key Decisions

- **Location Logic:** The app will fetch the user's location every 5 minutes and snap it to the closest building on the map using the campus dataset.
- **Session Management:** If a user navigates to a protected dashboard page without an active session, the app will now force a redirect to the login page immediately.
- **Event Feedback:** We decided that users can only leave reviews/comments for sessions that have officially ended in the database.

Task Assignments

- **Sean:** Finalize the Euclidean calculation logic for nearest-building identification and integrate it with the user profile.
- **Yash:** Add a "warning" UI component that appears when the auto-timeout has 2 minutes remaining.
- **Vaughn:** Fix the "missing authorization" bug in the frontend that was preventing chat history from loading for some users.
- **Aqsa:** Complete the MapView class debugging and ensure "upstream" pushes are restricted to the dev branch for now.
- **Omar:** Complete the database security rules audit and verify that "Reported" users are flagged correctly in the admin view.

Meeting Minutes: March 24, 2026 (48 min) Attendees: Bhavya, Vaughn, Yash, Omar, Sean, Aqsa

Summary Our final integration meeting was used to coordinate the merging of branches. We confirmed that the "Forgot Password" functionality is fully operational through Firebase. There was a discussion about whether navigation should be built-in or external; we decided to offer a choice between car, transit, and walking, which will then launch the device's native map application. We also verified that all members have begun their unit and integration testing.

Key Decisions

- **Navigation:** The app will showcase the route within the MapView but provide a "Open in Maps" button for turn-by-turn navigation (ID 48).
- **UI/UX:** Added a final "Logout" button to the Layout component so it is accessible from any subpage.
- **Final Testing:** All features are now under "Code Freeze." The next 48 hours will be dedicated exclusively to unit, integration, and customer tests.

Task Assignments

- **Vaughn:** Ensure all "Stub Repositories" used for testing are updated to work with the live Firestore Services.
- **Yash:** Complete unit tests for the 2FA code verification and password reset flow.
- **Bhavya:** Verify that the "Auth Fallback" in the chat has been removed to prevent unauthorized access.
- **Aqsa:** Fix the "blank space" UI bug on the MapView and finalize the navigation choice icons (Walk/Transit/Car).

- **Team:** Complete the final Item Log and prepare the code repository for the final submission.

Rationale of Changes on Plan and Big Design Decisions

Category	ITR2 (Before)	ITR3 (After)
Database Reads	Continuous polling for chat messages resulting in ~21k reads/day (42% of quota).	Adaptive, lightweight "check first, fetch if needed" metadata endpoint with polling jitter.
Authentication & Security	Partial auth protection; localStorage token checks only. Route guards did not validate tokens.	Centralized auth state (`auth.ts`). Strict backend token validation on protected routes; full state clearance on logout/inactivity.
Events & Accountability	Basic event creation, hosting, and joining.	Event reviews and ratings introduced; User Profiles now display an "Average Host Rating". Fixed cancellation permissions.
Data Standardization	Free text inputs for Majors, Courses, and Locations.	Standardized dropdown lists for degrees, courses, and constrained University locations.
Map & Navigation	Basic map view with hardcoded pin drops and current location.	Dynamic pinpoints with geographical boundary limits (preventing off-campus/invalid event creation).

User Engagement	Basic session attendance tracking.	Addition of a Study Session Log interface, an activity Streak Counter, and a secure Reporting system.
Inactivity Handling	Inactivity redirected users but left session/tokens active.	System-wide auto-timeout that comprehensively clears auth state and forces a true sign-out.
Testing	Unit tests required for all features; carried-over auth tests completed	Added more comprehensive unit, integration, and customer tests based on our added features.

1. Database Optimization: Polling to Adaptive Lightweight Fetching

- **ITR2 State:** In ITR2, we fully integrated Firestore to support real-time chat features. However, the client polled Firestore every few seconds for the last 21 messages. Each poll triggered a full read of those documents, even if no new messages existed, resulting in over 21,443 reads per day (consuming 42% of our daily quota on a single day).
- **Change in ITR3:** We replaced repeated full reads with a "check first, fetch if needed" architecture. We created a lightweight metadata endpoint that only returns the `latestTimestamp` and `latestMessageId`. The client now uses adaptive polling intervals (e.g., 3-5 seconds if active, 30-60 seconds if in the background) with a ±10% jitter to prevent backend load spikes. Full message payloads are only fetched if the metadata probe indicates a change.
- **Rationale:** The previous pattern scaled too aggressively and became prohibitively expensive. By maintaining minimal local state (`lastMessageId`, `lastTimestampEpochMillis`) and denormalizing chat fields, we drastically reduced our Firestore read footprint while maintaining the illusion of real-time responsiveness for the user.

2. Authentication Hardening & Session Cleanup (Security Optimization)

- **ITR2 State:** We established full domain-restricted auth (@yorku.ca) and email verification. However, protected routes only checked if a token string existed in the browser storage, not whether it was actually valid. Logout and inactivity timeouts only removed a single key, leaving stale auth-adjacent state behind.
- **Change in ITR3:** We implemented centralized auth-state management (`auth.ts`) and updated the `Login.tsx` and `Layout.tsx` components to use it consistently. Protected routes now actively validate the token against the backend profile endpoint. Furthermore, system-wide inactivity timeouts and manual sign-outs now execute a complete sweep of all auth state (`STUDY_BUDDY_TOKEN_KEY` and `DEV_ACTOR_STORAGE_KEY`).
- **Rationale:** This fixes a critical security loophole. Previously, incomplete session clearing could allow stale sessions or inconsistent behavior when navigating directly to protected

pages (like /chat) without proper login credentials. This optimization guarantees that direct protected-page access strictly depends on a continuously valid authenticated session.

3. Event Reviews, Ratings, and Permission Fixes

- **ITR2 State:** Users could host, join, and cancel events. However, there was a bug regarding who could cancel attendees, and no system existed to hold hosts accountable for the quality of their study sessions.
- **Change in ITR3:** We implemented Event Reviews (Details & Opening) and an Event Review Rating system. User profiles now calculate and display an "Average Host Rating". Additionally, we fixed the attendee cancellation logic: users can only cancel their *own* attendance, while the host retains the right to cancel/remove attendees.
- **Rationale:** Introducing a rating system improves community trust and safety by holding hosts accountable for their sessions. Fixing the cancellation permissions ensures users have proper authorization boundaries over their schedules and event rosters.

4. Data Standardization and Secure Reporting

- **ITR2 State:** User profiles relied on "free text" forms for critical data points like Majors, Courses, and Locations.
- **Change in ITR3:** We replaced free text fields with restricted dropdown lists (e.g., a full list of common degrees, specific courses, and university-only locations). We also implemented a secure database reporting system for community moderation.
- **Rationale:** Free text inputs lead to fragmented, messy data, making our matching algorithms and database queries highly inefficient. Standardizing these fields improves searchability, UI consistency, and data integrity. The reporting system addresses necessary safety and moderation requirements.

5. Advanced Map Constraints and Gamification

- **ITR2 State:** Map functionalities displayed available study sessions near the user, but lacked geographic restrictions.
- **Change in ITR3:** We implemented logic to limit where users can place events (e.g., restricting pinpoint creation to university boundaries, rather than allowing events in random global locations). We also introduced user gamification via a Streak Counter and detailed Study Session Logs.
- **Rationale:** Geographic limits maintain the integrity of the app, ensuring that it remains a hyper-local tool tailored for the York University campus and preventing spam events. The session logs and streak counters serve to incentivize consistent study habits and improve user retention.

Concern with the Project or Group Members

No concerns. We were able to manage our time and stay on top of ITR3 better this iteration.

Task Assignments and Estimated/Actual Time per Development:

User Story	Assignee(s)	Estimated Time	Actual Time	Status	Notes
0.1: Limit Firebase Reads/Writes	Bhavya	3 days	5 days	Completed	Reduce the amount of Firebase reads and writes by the app
0.2: Auth Cleanup	Bhavya	3 days	4 days	Completed	Revisit the login/log out flow and ensure credentials/sign-in tokens are properly removed during sign out. You should not be able to directly go to a /chat page without logging in.
0.3: Design-Level Refactoring	Vaughn	3 days	4 days	Completed	Improvements of the design with respect to code smells that are present.
1.2: View Study Session Log	Sean	3 days	2 days	Completed	Displaying a chronological history of past attended or hosted sessions, along with some study statistics for users.
1.8: Study Streak Counter	Sean	3 days	2 days	Completed	Display the consecutive days a user has logged in to the app. Resets upon not logging in for a day.

3.1. View possible sessions on map	Aqsa	1 day	1 day	Completed	Shows pinpoints of all events on the map.
3.7. Navigation on map to study session	Aqsa	2 days	3 days	Completed	Show navigation on the map to guide users from their current location to the designated study spot.
2.6: Privacy Policy - Events Restrictions	Vaughn	3 days	3 days	Completed	Description Seeing all other attendees of an event you joined. Not visible to users who have not joined the event.
2.7: Events Review Details	Vaughn	5 days	6 days	Completed	Allowing attendees to review an event once it finishes.
2.9: Events Review Comments	Vaughn	2 days	3 days	Completed	Seeing past reviews and commenting on them to interact.
3.6. Display current location	Sean	2 days	2 days	Completed	Track the current location of the user and display it on their profile.
4.3: User Reporting System	Omar	2 days	3 days	Completed	Using a reporting system to keep track of users/reports.
4.5: Auto-Timeout Feature	Yash	2 days	2 days	Completed	Automatically timing out the session after 5 to 30 minutes of inactivity (set by user)
4.6: Secure Database	Omar	3 days	4 days	Completed	Keeping the app secure as a whole with respect to database (user information).

ITR3 14 Mar – 27 Mar (14 work items)				Complete sprint
ID-100	0.1: Limit Firebase Reads/Writes	REVIEW	-	BT
ID-101	0.2: Auth Cleanup	REVIEW	-	BT
ID-103	0.3: Design-Level Refactoring	REVIEW	-	VC
ID-91	1.2: View Study Session Log	REVIEW	-	SL
ID-102	1.8: Study Streak Counter	REVIEW	-	SL
ID-33	3.1. View possible sessions on map	REVIEW	-	AM
ID-48	3.7. Navigation on map to study session	REVIEW	-	AM
ID-95	2.6: Privacy Policy - Events Restrictions	REVIEW	-	VC
ID-92	2.7: Events Review Details	REVIEW	-	VC
ID-94	2.9: Events Review Comments	REVIEW	-	VC
ID-46	3.6. Display current location	REVIEW	-	SL
ID-97	4.3: User Reporting System	REVIEW	-	OF
ID-98	4.5: Auto-Timeout Feature	REVIEW	-	YB
ID-62	4.6: Secure Database	REVIEW	-	OF
+ Create				

Development Tasks per User Story (Estimated/Actual Time):

0.1: Limit Firebase Reads/Writes

Task	Estimated Time	Actual Time
Identify unnecessary or repeated database reads/writes.	4 hours	3 hours
Implement caching or query optimization where possible.	3 hours	3 hours
Monitor usage to ensure	2 hours	3 hours

limits are not exceeded.		

0.2: Auth Cleanup

Task	Estimated Time	Actual Time
Remove unused or redundant auth code.	2 hours	1 hour
Refactor authentication flow for clarity.	1 hour	2 hours
Ensure consistent error handling.	2 hours	2 hours
Verify authentication works across all protected features.	2 hours	3 hours

0.3: Design-Level Refactoring

Task	Estimated Time	Actual Time
Refactor classes/services for better separation of concerns.	3 hours	4 hours
Improve naming and code organization.	2 hours	2 hours
Ensure changes do not break existing features.	2 hours	4 hours

1.2: View Study Session Log

Task	Estimated Time	Actual Time
Retrieve completed study sessions for the user.	2 hours	3 hours
Display session history in a dedicated window.	3 hours	5 hours

Show key details (course, date, duration).	2 hours	4 hours

1.8: Study Streak Counter

Task	Estimated Time	Actual Time
Define rules for calculating a study streak.	2 hours	4 hours
Calculate consecutive study days.	2 hours	3 hours
Display the streak on the user profile and dashboard.	3 hours	2 hours

3.1. View possible sessions on map

Task	Estimated Time	Actual Time
Retrieve available study sessions from the database.	3 hours	4 hours
Display session locations as markers on the map.	4 hours	4 hours
Allow users to select markers to view session info.	3 hours	3 hours

3.7. Navigation on map to study session

Task	Estimated Time	Actual Time
Allow users to select a study session on the map.	3 hours	4 hours
Generate a visual route from current location to the session.	4 hours	5 hours
Display directions on the map.	3 hours	2 hours
Update route if the user location changes.	2 hours	5 hours

2.6: Privacy Policy - Events Restrictions

Task	Estimated Time	Actual Time
Restrict joining based on event privacy settings.	3 hours	4 hours
Create and store the friend request in the database.	2 hours	3 hours
Store privacy settings in the database.	3 hours	2 hours

2.7: Events Review Details

Task	Estimated Time	Actual Time
Display a summary of event details before creation or joining.	4 hours	3 hours
Show key info (location, time, participants).	4 hours	4 hours
Allow users to confirm or go back to edit.	2 hours	3 hours

2.9: Events Review Comments

Task	Estimated Time	Actual Time
Create a comment section for events.	4 hours	3 hours
Allow users to post and view comments.	2 hours	5 hours
Store comments in the database.	3 hours	4 hours
Store comments in the database.	2 hours	2 hours

3.6. Display current location

Task	Estimated Time	Actual Time
Request location permission from the user.	3 hours	2 hours
Retrieve current geolocation coordinates.	2 hours	3 hours
Allow manual input if the user rejects.	2 hours	3 hours
Have paths for “allow once”, “allow always”, “reject”.	2 hours	4 hours

4.3: User Reporting System

Task	Estimated Time	Actual Time
Add a “Report User” option on profiles.	3 hours	4 hours
Allow users to select a reason and submit a report.	3 hours	2 hours
Store reports in the database for review.	2 hours	3 hours
Store reports in the database for review.	2 hours	3 hours

4.5: Auto-Timeout Feature

Task	Estimated Time	Actual Time
Detects user inactivity after a set time.	4 hours	5 hours
Trigger automatic logout after timeout.	3 hours	3 hours

AI Usage Declaration:

Prompt:

Review my EventService class and identify potential edge cases that might not be covered by my current test suite.

Response:

The AI highlighted several overlooked scenarios, such as events with missing host IDs, events created without a location, invalid date formats, and concurrency issues when multiple users join an event simultaneously. It also suggested adding tests for null attendees lists, malformed event IDs, and Firestore failures.

Rationale:

These suggestions helped me expand my test coverage beyond the obvious cases. Some of the edge cases were ones I had not initially considered, and incorporating them made the backend more robust.

Prompt:

How do I design a clean DTO mapping strategy for converting Firestore documents into EventResponseDTO objects?

Response:

The AI recommended using a dedicated mapper class with static factory methods, separating Firestore entity parsing from controller logic. It also suggested validating fields like host, attendees, and timestamps before constructing the DTO.

Rationale:

This guidance helped me refactor the code to be more maintainable and modular. The mapper approach reduced duplication and made the integration tests easier to write.

Prompt:

Help me understand how to structure pagination for chat messages in a scalable way.

Response:

The AI explained cursor-based pagination using message timestamps and described how to store and query messages efficiently. It also recommended returning metadata such as hasMore and nextCursor to the frontend.

Rationale:

This explanation clarified the architectural approach and influenced how I implemented message syncing and pagination in the chat module.

Prompt:

Generate example Firestore security rules that would protect user data while still allowing chat and event features to function.

Response:

The AI produced a set of rules enforcing that users can only read/write their own profile data, can only join events that exist, and can only send messages to chats they are part of. It also recommended validating fields like senderId and chatId in write rules.

Rationale:

Although we did not directly use these rules due to the stubbed environment, the examples helped us understand how real Firestore security would work and guided our backend authorization logic.

Prompt:

Help me design a clean error-handling strategy for the backend so that all controllers return consistent error responses.

Response:

The AI suggested implementing a global exception handler using @ControllerAdvice, defining custom exception types, and returning standardized JSON error objects with fields like message, timestamp, and statusCode.

Rationale:

This helped us unify error handling across the project and made integration tests more predictable and consistent.

Prompt:

Help me write documentation for the chat architecture, specifically how typing indicators, message syncing, and presence detection interact.

Response:

The AI generated a structured explanation describing the flow between frontend → backend → stub database, including diagrams and sequence steps. It also suggested how to document the API endpoints and their expected payloads.

Rationale:

This documentation made it easier for the team to understand the chat subsystem and ensured that new features were added consistently.

Prompt:

Propose a strategy for migrating from the stub database to Firestore without breaking existing endpoints.

Response:

The AI recommended introducing repository interfaces, implementing both stub and Firestore versions, and using Spring profiles to switch between them. It also suggested writing integration tests for both implementations.

Rationale:

This aligned with our development goals and helped us plan a smooth transition path while keeping the codebase maintainable.